

BVR RL Project Guide

Training an AI pilot for beyond-visual-range air combat with reinforcement learning: how to install it, run it, read it, and how every part works.

BVR_RL · branch feature/platforms-2v1 @ a5b8cb8 · 7 October 2026
Everything in this guide describes the code at this commit. Simulation revision 8.

Contents

Part A · Getting started		14. Two against one (2v1)	17
1. What this project is	4	15. The parameter library	18
2. Installing and starting	5	Part C · The GUI, page by page	
3. Your first training run	6	16. The TRAINING page	21
4. Watching a trained model	7	17. The WATCH page	22
5. Comparing models with cross-play	7	18. The CROSS-PLAY page	23
Part B · How it works		19. The LIBRARY page	23
6. The simulated world	10	Part D · Working with results	
7. One episode, step by step	11	20. Reading a training run	25
8. What the agent sees	12	21. Symptoms and what to do	25
9. What the agent can do	13	22. Files, checkpoints and versions	26
10. How the agent is rewarded	13	23. Command-line reference	27
11. How the agent learns	15	24. DCS World: first step	28
12. The curriculum and the opponents	15	25. Troubleshooting	28
13. Self-play	16	26. Glossary	29

How to read this guide

- **New to the project?** Read Part A from start to finish and do the steps. You'll have trained, watched and compared a model.
- **Want to understand what the agent is doing?** Part B explains the simulation, the agent's inputs and actions, the reward, and the training method.
- **Looking for a field or a number on screen?** Part C explains every page of the GUI.
- **Something looks wrong?** Part D lists the symptoms seen in this project, their causes and what to do.

PART A

Getting started

Install the project, train a first model, watch it fly, and compare it with other models.

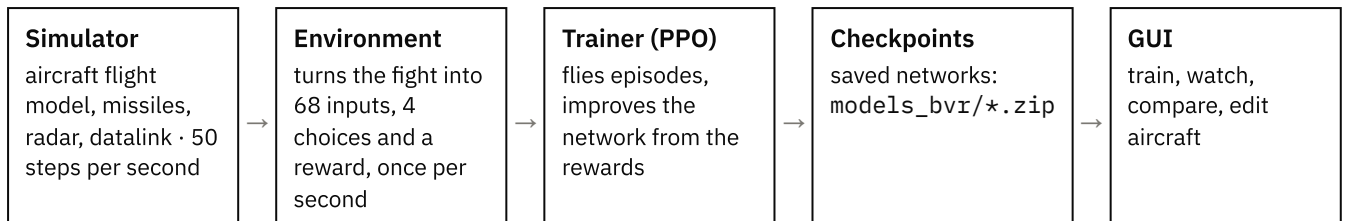
1. What this project is
2. Installing and starting
3. Your first training run
4. Watching a trained model
5. Comparing models with cross-play

1 · What this project is

BVR RL trains an artificial pilot for **beyond-visual-range (BVR) air combat**: engagements where the two aircraft are tens of kilometres apart, see each other only on radar, and fight with radar-guided missiles of the AIM-120 class.

The pilot is a neural network. It is not programmed with tactics. It learns them by flying hundreds of thousands of simulated engagements and being rewarded for good outcomes. This is called **reinforcement learning**.

The pieces



Piece	Main files	What it does
Simulator	f16_sim.py, missile_sim.py, sim_world.py, bvr_radar_sim.py, bvr_track_adapter.py	Flies the aircraft and missiles and models what each aircraft's radar sees.
Environment	bvr_env.py (1v1), bvr_team.py (2v1)	Starts each engagement, builds the agent's inputs, applies its choices, computes rewards, decides when the fight is over.
Opponents	bvr_opponents.py, bvr_selfplay.py	Scripted enemy pilots of increasing difficulty, and frozen copies of the agent for self-play.
Trainer	train_bvr.py, bvr_policy.py	Runs PPO training, the curriculum, self-play snapshots, and writes metrics for the GUI.
Evaluation	crossplay.py	Plays saved models against each other and scores them.
Library	bvr_library.py, library/	Every number for each aircraft, radar and missile, editable and versioned.
GUI	bvr_gui.py, bvr_gui.html	A browser interface to all of the above.
Tests	test_sim.py	36 checks of the simulator, environment, rewards and tools.

What the agent learns

- **Find and hold the enemy on radar.** Its radar sees only $\pm 60^\circ$ off the nose, out to a range that depends on the enemy's size.
- **Shoot at the right moment.** Too far and the missile runs out of energy; too close and the enemy shoots first.
- **Support its missile.** Its missile needs radar updates until its own seeker locks on.
- **Defend.** Turn away or across an incoming missile to make it miss.
- **Manage energy and altitude.** Speed and height extend a missile's reach.

Formats

- **1 v 1:** one agent aircraft against one enemy. This is the main format.
- **2 v 1:** two agent aircraft, flown by the same network, against one enemy. They share radar tracks over a datalink (chapter 14).

2 · Installing and starting

What you need

- A computer with Python 3.10 or newer. The project is used on Windows; it also runs on Linux.
- About 2 GB of disk space for Python packages, plus room for models (each about 5 MB).
- More CPU cores make training faster: each parallel environment uses one core.
- A modern web browser for the GUI.

Install, step by step

- 1 **Get the code.** Clone the repository and switch to the working branch:

```
git clone https://github.com/dursunbekci/BVR_RL.git
cd BVR_RL
git checkout feature/platforms-2v1
```

- 2 **Install the Python packages.**

```
pip install numpy torch gymnasium stable-baselines3 sb3-contrib tensorboard websockets
matplotlib
```

The project was last tested with numpy 2.4, torch 2.14, gymnasium 1.3, stable-baselines3 and sb3-contrib 2.9, websockets 17.1, tensorboard 2.21.

- 3 **Check everything works.** Run the test suite. It takes a few minutes and should end with 36/36 passed.

```
python test_sim.py
```

- 4 **Start the GUI.**

```
python bvr_gui.py
```

Then open **http://localhost:5006** in your browser. The GUI uses two ports: 5006 for the page and 5010 for live updates.

Getting updates

When new code is pushed, stop the GUI (Ctrl+C), run `git pull`, and start it again. A training run that is already going keeps the code it started with; the changes apply to the next run.

What the GUI looks like

Four tabs across the top:

Tab	Use it to
TRAINING	Set up and start a training run, then follow its progress on the dashboard.
WATCH	Load a saved model and watch it fly engagements in a 3-D view, with statistics.
CROSS-PLAY	Play several saved models against each other and rank them.
LIBRARY	Browse and edit the aircraft, radar and missile parameters.

The button to the right of the tabs switches units between metric (m, km, m/s) and aviation units (ft, nm, kt). Every field has a ? button that explains it.

3 · Your first training run

This run trains an F-16 against an F-16, starting with the easiest opponent. Everything happens on the **TRAINING** tab.

- 1 **Engagement.** Set the format to **1 v 1**, AGENT FLIES to **F-16C**, OPPONENT FLIES to **F-16C**.
- 2 **Opponent stage.** Choose **STRAIGHT**. This is the first stage of the curriculum: an enemy that flies straight and never shoots.
- 3 **Missile doctrine.** Leave it at **MIXED**. Each episode then uses one of three costs per missile fired, and the agent learns all three (chapter 10).
- 4 **Training steps.** Start with **2,000,000**. One step is one decision, which is one simulated second. The estimate under the field shows how long that takes on your machine.
- 5 **Parallel envs.** Choose as many as you have spare CPU cores (4 or 8 on most desktops). Each runs its own simulation in a separate process.
- 6 **Gamma, learn rate, batch.** Leave the defaults: 0.997, 0.00025, 256 (chapter 11 explains them).
- 7 **Auto-curriculum.** Leave it **ON**. The opponent then gets harder by itself as the agent improves (chapter 12).
- 8 **Model name.** Give the run its own name, e.g. `f16_first`. The final model is saved as `models_bvr/f16_first.zip`.
- 9 **Resume from.** Leave it at **fresh start**.
- 10 **Start.** Click ► **START TRAINING**. The first episode may take 30–60 seconds while the network is built.

What you'll see

- **The log** prints a line every 10 episodes: win, loss, mutual and timeout shares, hit rate (Pk), shots per episode and more.
- **The dashboard** fills in after the first 10 episodes: key numbers along the top and charts below. Chapter 16 explains each one.
- **The curriculum bar** at the top shows the current stage and how close the agent is to advancing.

Stopping and saving

- 1 Click ■ **STOP** at any time. The trainer saves the model before it exits; wait for "saved final model" in the log.
- 2 The model appears as `models_bvr/<model name>.zip`. Along the way, the trainer also saves backups in `models_bvr/archive/`: the best model at each stage, and the model just before each stage change (chapter 22).
- 3 To continue later, start a new run with **RESUME FROM** set to that file, and give the new run a new model name.

What a good first run looks like

Against STRAIGHT, the win rate should climb from near 0 to over 65% within a few hundred thousand steps, and the curriculum then moves on to EVASIVE. The track fraction should rise and timeouts should fall. If the win rate stays flat for a million steps, see chapter 21.

Long runs

For an overnight run, work out the steps from your machine's rate. Example from this project: 92 episodes a minute at about 210 steps each is about 1.16 million steps an hour, so 12 hours is about 14 million steps. Resume from your last model, keep the same number of parallel envs, and give the run a new name so it doesn't overwrite the previous model.

4 • Watching a trained model

The **WATCH** tab flies a saved model live, in a 3-D view, and sums up the episodes it plays.

- 1 **Checkpoint to load.** Pick the model, e.g. `f16_first.zip` or an archive file. The list is read live from `models_bvr/`.
- 2 **Opponent.** Pick who it fights. It doesn't have to be the opponent it trained against: e.g. train against STRAIGHT, then watch against SHOOTER to see what transfers.
- 3 **Playback speed.** 3× is good for following tactics, 10× or 50× for scanning many episodes.
- 4 **Episodes.** How many to play before stopping by itself: 10, 25 (default), 50, 100, 200, or until stopped.
- 5 **Missile doctrine.** The missile cost the model is told about (BALANCED by default).
- 6 **Opponent flies** and **Format.** By default, the platform and format the model was trained with.
- 7 Click ► **START EVAL.**

What you'll see

- **The 3-D tactical view** shows both aircraft, their trails and missiles. Blue is the agent, red the opponent.
- **Live engagement** shows range, closure, track state, missile status and more for the current moment.
- **Watch statistics** sums up the session: outcomes, win rate with its 95% range, exchange ratio, hit rate, shots, shot range, and win/loss by start geometry and start range (chapter 17).
- **Shot Trace** ( in the log bar) lists every missile launch of the last episode: range, aspect, track quality, and where in the envelope it was fired.

WATCH is not the training dashboard

WATCH always takes the model's single most likely action. During training the model samples its actions, so it sometimes tries less likely ones. WATCH numbers can therefore differ from the training dashboard, usually upwards. For a proper comparison of models, use CROSS-PLAY.

5 • Comparing models with cross-play

CROSS-PLAY plays every chosen model against every other, in both seats, and optionally against scripted opponents. It runs without drawing, in parallel, so it's much faster than WATCH and gives fair, comparable numbers.

- 1 **Checkpoints.** Tick the models to compare. Quick links select all, only the archive, or none.
- 2 **Scripted columns.** Optionally add scripted opponents (e.g. SHOOTER, ADAPTIVE) as extra columns, to see how each model fares against them.
- 3 **Games per cell.** 100 is a good default. More games give narrower uncertainty ranges.
- 4 **Workers.** Parallel processes; use your spare CPU cores.
- 5 **Doctrine.** Both sides fly the same missile doctrine.
- 6 Click ► **RUN CROSS-PLAY.** The estimate under the fields shows the number of games and the expected time.

Reading the results

- **The matrix:** each cell is the **row** model's score against the **column** model, with the row model in seat AC1. A win scores 1, a draw (mutual kill, timeout, escape) 0.5, a loss 0. Green is above 0.5, red below. The $\pm$ value is the 95% uncertainty range.
- **Head-to-head between two models:** average both cells. For example, if v5 scores 0.67 against v4 and v4 scores 0.56 against v5, v5's score is $(0.67 + (1 - 0.56)) / 2 \approx 0.56$.
- **Elo ratings** rank all models on one scale with a mean of 1000. A 100-point gap means the higher model scores about 0.64 against the lower one.
- **Seat bias** is how far the AC1 seat scores above 0.5 when the same starts are played both ways. It cancels in head-to-head scores and Elo, since every pairing is played from both seats (chapter 18).

Every pairing starts from the same geometries

Game i of every pairing starts from the same geometry, so differences between cells come from the models, not from the luck of the draw.

PART B

How it works

The simulation, what the agent sees and can do, how it is rewarded, how it learns, and the opponents it learns against.

- 6. The simulated world
- 7. One episode, step by step
- 8. What the agent sees
- 9. What the agent can do
- .0. How the agent is rewarded
- .1. How the agent learns
- .2. The curriculum and the opponents
- .3. Self-play
- .4. Two against one (2v1)
- .5. The parameter library

6 • The simulated world

Everything runs in Python, 50 times per simulated second. The agent decides once per second; between decisions, the simulator flies its last command.

6.1 Aircraft

Each aircraft is a point-mass flight model with an autopilot. The agent does not move the stick: it chooses a **heading**, a **target altitude** and a **speed**, and the autopilot flies there within the aircraft's limits (turn rate, climb angle, thrust, drag, fuel). Two platforms ship with the project:

Platform	Speed choices (m/s)	Missiles	Radar size (RCS)	Operating altitude
F-16C	220 · 280 · 340 · 400	4	1.2 m ²	1,000–14,000 m
GENERIC-UCAV	170 · 200 · 225 · 250	2	0.1 m ²	1,000–12,500 m

The generic UCAV is illustrative, not data for a real aircraft. Both are defined in the parameter library (chapter 15).

6.2 Missiles

The missile model is of the AIM-120 class. A missile's life:

- 1 **Boost** (6 s): the motor burns and the missile accelerates.
- 2 **Midcourse**: it flies towards where the shooter's radar says the target is, updated over a datalink. **If the shooter stops sending updates for 3 seconds, the missile is lost** (the "3-second support rule").
- 3 **Hand-off**: within 8–16 km of the target (less against a small target), its own seeker locks on. From then on it no longer needs the shooter.
- 4 **Terminal**: it steers itself to the target, pulling up to 30 g. It kills inside a 10 m radius, misses if it runs out of energy, and is gone after 120 s at most.

Launch envelope. How far a missile can reach depends on the shooter's speed and altitude and on the target's direction of flight and altitude. Head-on, the target flies into the missile, so the reach is longest; tail-on it is several times shorter. A target far below makes the missile end its flight in thick air: from 9 km, the AIM-120C-DCS reaches 84 km head-on against a level target, 58 km against one at 1 km (target altitude is measured since SIM_REV 11). The project measures this once per missile type with `sweep_envelope.py` and stores it as a table. Two numbers come out of it:

- **R_{max}**: the farthest range at which a shot can hit a target that keeps flying as it is.
- **R_{NEZ}** (no-escape zone): the range inside which a shot hits even if the target turns and runs at once.

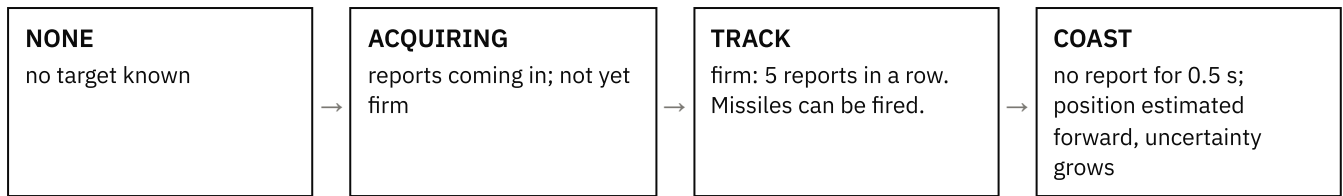
6.3 Radar

Each aircraft has a radar model that behaves like a radar that has already locked onto the target. It does not search or scan.

- **Coverage**: $\pm 60^\circ$ off the nose, left/right and up/down.
- **Range**: the platform's maximum range (90 km for the F-16C's radar, 70 km for the UCAV's) scaled by the target's size: $\text{range} \times (\text{target RCS} / 1.2 \text{ m}^2)^{3/4}$. A small UCAV (0.1 m²) is seen at only about 54% of the range of an F-16.
- **Errors** grow with the square of range, are about 4 times larger when the target flies across the line of sight (low closure, the "Doppler notch"), and drift smoothly rather than jumping (about 2 s correlation).
- **Rate**: 10 reports per second.

6.4 Track

The radar reports are turned into a **track**: an estimate of the target's position and velocity, with an uncertainty. The track has four states:



- A new report brings a COAST track back to TRACK.
- After 15 s without reports, the track is dropped (NONE).
- If the position uncertainty grows beyond 2 km, the track falls back to ACQUIRING.

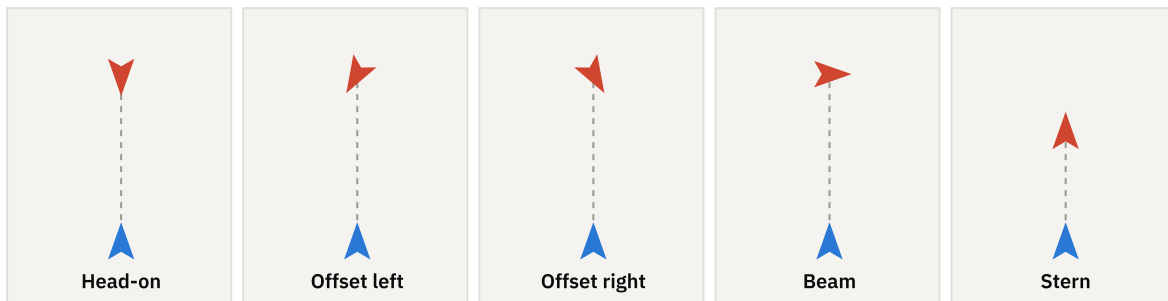
6.5 What the opponent knows

Scripted opponents see the true positions: they never lose track. That is deliberately in the agent's disfavour, so that beating them is a real result. In self-play, the opponent copy has its own radar and track, exactly like the agent.

7 • One episode, step by step

7.1 The start

Each episode starts with the two aircraft 70–110 km apart, at 6–11 km altitude (within 2.5 km of each other), at 250–320 m/s (scaled to slower platforms). One of five start geometries is drawn:



The five start geometries, drawn for AC1 (blue, bottom) and AC2 (red, top). AC1 always starts pointing at AC2. In a stern start AC2 flies away and the range is shorter, 45–75 km. Stern starts are skipped when the opponent is faster than the agent, since they could never be won.

Mirrored starts. In four of the five geometries, AC1 has the better start. So in half the episodes the two aircraft swap starts: the opponent gets the better one. The agent learns to fight from both sides. WATCH statistics mark mirrored starts with $\rightleftharpoons$.

7.2 Each second

- 1 The agent receives its 68 inputs (chapter 8), together with the previous 7 seconds' inputs.
- 2 It chooses a heading, an altitude change, a speed and whether to fire (chapter 9).
- 3 The simulator flies 50 small steps (1 s) with that command. The radar reports 10 times, the opponent acts, missiles fly.
- 4 The agent receives the reward for that second (chapter 10).

7.3 The end

Outcome	When
KILL	The opponent is destroyed by the agent's missile.
SHOT_DOWN	The agent is destroyed by the opponent's missile.
MUTUAL_KILL	Both are destroyed. A kill ends the fight only once no missile is still in flight at a live aircraft, so a shooter that is hit can still take its killer with it.
TIMEOUT	300 decisions (300 s) pass with no result. The limit is MAX_STEPS in <code>bvr_env.py</code> ; slow platforms may need more (chapter 21).
ESCAPE	The range grows beyond 150 km.
CRASH	The agent flies below 300 m.
BANDIT_CRASH	The opponent flies below 300 m.

8 · What the agent sees

Every second the agent receives **68 numbers**. Each is scaled to the range -1 to $+1$. The agent also gets the previous 7 seconds' numbers (8 frames in all), so it can see how things are changing.

Important: everything about the enemy comes from **its own radar track**, never from the true position. If the track is poor, the agent's picture is poor.

Group	Inputs	What they are
Own aircraft	10	Mach, altitude, climb angle, bank angle, load factor, energy, angle of attack, missiles left, fuel.
Track	14	Track state (NONE / ACQUIRING / TRACK / COAST), range, closing speed, angle off the nose (left/right and up/down), the enemy's aspect (which way it points relative to us), track age, position and velocity uncertainty.
Enemy estimate	4	Enemy speed, altitude difference, energy difference, line-of-sight rate.
Envelopes	4	Range $\div$ own $R_{\max}$, range $\div$ own R_{NEZ} , and the same for the enemy's missile against us. Below 1 means "inside".
Own missiles	6	Missiles in flight, nearest one's time-to-go, time in flight, whether it still needs support, whether its seeker is active, time since its last update.
Threat warning	6	Missile launch warning, how long ago, its bearing, whether its seeker is active, time-to-go of the nearest inbound missile.
Time and doctrine	3	Fraction of the episode elapsed, time since the last shot, the doctrine's cost per missile.
Wingman (2v1)	17	The wingman's position, heading, speed, track, missiles and threats. All zero in 1v1.
Heading	4	The previous heading choice and the turn still to fly.
	68	

The critic sees more

The network has two halves (chapter 11). The **actor** chooses actions and sees only the 68 inputs above. The **critic** judges how good a situation is, and during training also gets **24 true values**: true range, closure, aspects,

energy, the enemy's missiles and so on. This helps it learn faster. The critic is not used when the model flies, so the agent never relies on information it couldn't have.

9 · What the agent can do

Once per second the agent makes four choices at once:

Choice	Options	Meaning
Heading	0, ± 30 , ± 50 , ± 90 , ± 135 , 180°	An offset from the bearing to the enemy. 0 = straight at it, ± 50 = crank, ± 90 = beam, 180 = straight away. Positive = to the right. With no track, the offset is from the last known bearing.
Altitude	-3000, -1200, 0, +1200, +3000 m	Change of target altitude from the current one, kept inside the platform's operating band.
Speed	the platform's 4 speeds	F-16C: 220, 280, 340, 400 m/s.
Fire	no / yes	Launch a missile this second.

The fire rule

"Fire" can only be chosen when all of these hold. Otherwise it is **masked**, meaning the network cannot pick it:

- 1 Missiles are left.
- 2 The track is firm (TRACK state).
- 3 At least 3 s have passed since the last launch.
- 4 The target is within **60° of the nose**. The missile leaves along the aircraft's axis and must turn; beyond 60° it loses most of its reach.
- 5 The range is between **1.5 km** and $R_{\max}$.

Inside that window, choosing *when* to fire is up to the agent. The reward encourages shots at 0.6–0.85 of $R_{\max}$, where hits are likely.

Changing heading costs a little

Each change of the heading choice costs 0.04 per 180° of change (chapter 10). Without it, policies flipped between two similar choices every few seconds, rocking their wings. A deliberate defensive turn still costs very little.

10 · How the agent is rewarded

The reward is what the agent tries to maximise. It has three parts.

10.1 Outcome (last second only)

Outcome	Reward	Why
KILL	+1.0	The goal.
SHOT_DOWN	-1.0	The worst result.
MUTUAL_KILL	-0.4	Trading aircraft is bad, but better than only losing.

TIMEOUT	-0.5	Avoiding the fight is not a solution.
ESCAPE	-0.6	Running away is worse than a timeout.
CRASH	-1.0	Flying into the ground is as bad as being shot down.
BANDIT_CRASH	+0.4	The opponent crashed, often while defending.
Each wasted missile	-0.03	Every missile that did not make the kill.

10.2 Shaping (every second)

An episode can last 300 seconds, and the outcome comes only at the end. To help the agent learn which seconds mattered, every second also gets a small **shaping** reward: the change in a score Φ ("potential") of how good the situation is. Because it is only the *change*, it speeds up learning without changing what the best tactics are.

```
reward each second =  $\gamma \cdot \Phi(\text{after}) - \Phi(\text{before}) - \text{heading cost} - \text{shot cost}$ 
```

Term	Weight	Rewards
Envelope	0.60	Being inside your own missile's envelope (best inside R_{NEZ}) and outside the enemy's.
Track	0.25	Holding a firm track (1), acquiring (0.5), coasting (0.4, fading with age), none (0).
Energy	0.15	Having more energy (speed and height) than the enemy.
Support	0.30	Keeping a good track while your own missile still needs guidance.
Defence	0.80	Negative while an enemy missile is inbound, more so as its time-to-go runs out: $-0.8 \times (1 - \tanh(\text{time-to-go} / 25 \text{ s}))$. Defending lengthens time-to-go.

10.3 Costs

Cost	Size	Purpose
Heading change	0.04 per 180° of change	Stops wing-rocking. Flipping +30° ↔ -30° costs 0.013; a 135° defensive turn 0.03.
Shot (doctrine)	per missile fired: AGGRESSIVE 0 · BALANCED 0.10 · CONSERVATIVE 0.30	A price per missile. The agent sees the price as an input, so one model learns all three styles. MIXED training draws one per episode.

10.4 2v1 team rewards

In 2v1 both aircraft share a team reward: **+1.0** when the enemy is destroyed, **-1.0** for each blue aircraft shot down, -1.0 for a crash, -0.5 for a timeout, -0.6 for an escape, +0.4 if the enemy crashes. Each aircraft also gets its own shaping and costs.

Where to see it

On the TRAINING page, **REWARD DRIVERS** shows how much each shaping term and cost has moved the reward recently. A term near zero isn't influencing learning; a dominant one may be steering it too much.

11 • How the agent learns

11.1 The method: PPO, in plain words

- 1 **Fly.** Each parallel environment flies 1,024 decisions with the current network, recording inputs, choices and rewards.
- 2 **Judge.** The critic estimates how good each situation was. Comparing that with what actually followed shows which choices turned out better or worse than expected.
- 3 **Improve.** The actor is nudged to make the better choices more likely, in 6 passes over the data in batches of 256. PPO limits each nudge (the "clip") so that one batch can't change the network too much.
- 4 **Repeat** until the training steps are used up.

11.2 Settings

Setting	Value	Meaning
Gamma (γ)	0.997	How much future rewards count. A missile flies about 40 s, so a launch decision is 40 steps from its result. At 0.997, 89% of the result reaches the launch; at 0.99 only 67%; at 0.95 13%.
Learning rate	0.00025	Step size of each update. Lower if updates are unstable.
Steps per update	1,024	Per environment. Long enough to cover a whole engagement.
Batch size	256	Decisions per gradient step.
Passes	6	Passes over each batch of data.
Clip	0.2	Limit on how far one update may change the choice probabilities.
Target KL	0.02	Stops an update early if it changes the network too much.
Entropy	0.01	Keeps the agent exploring.
Network	256 × 256	Two hidden layers each for the actor and the critic.
Frames	8	The last 8 seconds of inputs are given together.

11.3 Masks

Impossible choices are removed before the network chooses (for example, firing without a firm track). This is "MaskablePPO". The agent never wastes time learning that impossible choices don't work.

12 • The curriculum and the opponents

Starting against a skilled opponent, the agent would rarely win and learn little. So it starts easy, and the opponent gets harder as it improves:



Stage	Armed?	Behaviour	What the agent learns
STRAIGHT	no	Flies straight at constant speed and altitude.	Find, track and shoot.

EVASIVE	no	Slow S-turns ($\pm 35\text{--}70^\circ$, every 25–45 s) and altitude changes.	Keep the radar on a changing target.
NOTCHER	no	When spiked or within 90 km, turns 90° to the agent (beaming) and descends 2 km, switching sides every 12–25 s.	That a track isn't permanent; how to close on a beaming target.
SHOOTER	yes	Commits; fires at 80% of its $R_{\max}$; cranks 45° while its missile needs guidance; turns cold and runs once it doesn't; re-attacks beyond 60 km; defends (beam and dive) against inbound missiles.	Fight an armed enemy: shoot, support, defend.
ADAPTIVE	yes	SHOOTER with random settings each episode: shot at 60–95% of $R_{\max}$, crank $30\text{--}45^\circ$, re-attack 30–55 km, 8–20 s between shots. Against an inbound missile, half its episodes it dives to 1–2.5 km instead of descending 1.5 km: beaming (as the DCS AI does) or turning away. A third of its episodes it flies like the DCS AI: it shoots at the edge of $R_{\max}$ and presses on after its missile goes active, so it often shoots first.	Not to overfit one opponent's habits.
SELF-PLAY	yes	Frozen copies of the agent itself (chapter 13).	Beat an opponent as good as itself.

In 1v1, SHOOTER and ADAPTIVE fly under the agent's rules (since SIM_REV 9): each runs its own copy of the radar model, aims its missiles at its own track and fires only when the agent would be allowed to. A red that turns too far off loses its track, and its missile misses after 3 s without support, as the agent's would. Before, red's missiles were guided by the true position, so red could crank wider and still hit. The 2v1 opponents are unchanged.

The deep dives (SIM_REV 10) matter because of the missile's energy: after its motor burns out it coasts, and the thick low air drains it. Against a level AIM-120C-DCS shot from 9 km, a red that turns away and dives escaped shots from 29 km that hit every beaming target. No ground clutter is modelled, so diving doesn't hide a target from the radar.

When does it advance?

The curriculum moves on when **all three** hold:

- 1 At least **60 episodes** have been played at the current stage (MIN EPISODES / STAGE). This makes sure the 50 episodes it judges are all against the current opponent.
- 2 The win rate over the **last 50 episodes** is at least **65%** (ADVANCE AT).
- 3 It has stayed there for **5 checks in a row**, one check every 10 episodes (HOLD FOR). A single lucky streak isn't enough: with 1 check, an agent that really wins 55% passed a 65% mark within 1,000 episodes 97% of the time; with 5 checks, 29%.

All three are set per run on the TRAINING page. For a mismatched matchup (e.g. two UCAVs against an F-16), 65% may be out of reach: lower ADVANCE AT.

At each advance, the model just before the change is saved as `archive/pre_<STAGE>.zip`. The best model at each stage is saved as `archive/best_<STAGE>_<win rate>.zip`.

13 • Self-play

In self-play the opponent is a frozen copy ("snapshot") of the agent itself. It only exists in 1v1 with both sides on the same platform, since a copy of the agent can only fly the aircraft it was trained on.

Step by step

- 1 **The pool starts empty.** At the start of every training run, the snapshot pool (`models_bvr/selfplay_pool/`) is emptied, so a run only ever meets copies of its own policy.
- 2 **First snapshot.** When the SELF-PLAY stage begins, the current policy is saved into the pool.
- 3 **More snapshots.** Every 250,000 steps another is added. The pool keeps the newest 12, about the last 3 million steps of the policy.
- 4 **Each episode, an opponent is drawn:**
 - 1 in 3 episodes: the scripted ADAPTIVE shooter, so a committed attacker stays in the mix. Without it, self-play tends to settle into both sides turning away at long range.
 - Otherwise, half the time the newest snapshot, and otherwise a random snapshot from the pool. Facing older versions stops the agent forgetting how to beat them.
- 5 **The snapshot flies fairly.** It has its own radar and track, the same inputs, the same firing rule, its own missile guidance, and its own doctrine.
- 6 **Only the agent learns.** Snapshots are frozen.

Watching it

- The log and TensorBoard split the win rate into **against snapshots** and **against scripted**. Around 50% against snapshots means parity with its recent selves.
- If the win rate against scripted falls while the rate against snapshots rises, the agent is overfitting to its own style: raise the scripted share.
- SELF-PLAY is the last stage; the curriculum doesn't advance beyond it.

Settings (SELF-PLAY section of the TRAINING page)

Setting	Default	Change it when
Scripted share	33%	Raise if the agent overfits to its own style; lower if the scripted shooter is already beaten easily.
Newest share	50%	Lower if it forgets how to beat older versions; raise for harder, more current opposition.
Snapshot every	250k steps	Shorter keeps opponents close to the current policy; longer spreads them over more of its history.
Pool size	12	Bigger keeps older opponents longer, at the cost of disk and loading time.

14 · Two against one (2v1)

In 2v1 two blue aircraft fight one red. Both blue aircraft are flown by **the same network**; each gets its own inputs and makes its own choices.

- **Datalink.** If an aircraft has no firm track of its own, it uses its wingman's, with 150 m extra uncertainty and 0.2 s delay. It can even fire on it.
- **Wingman inputs.** 17 of the 68 inputs describe the wingman: position, heading, speed, track, missiles, threats.
- **Formation.** The wingman starts beside the lead (abreast), behind it (trail) or behind and to the side (echelon).

- **Red's targeting.** Red opens on the nearer blue aircraft and switches only if the other gets much closer. While cranking, SHOOTER and ADAPTIVE also fire at the other blue aircraft if it is in range and within 60° of red's nose.
- **Rewards.** Team rewards (chapter 10.4). A mutual kill (red destroyed, one blue lost) is worth 0, between a clean kill (+1) and a timeout (−0.5).
- **No self-play yet.** Red is always scripted, so the 2v1 curriculum ends at ADAPTIVE.

Lessons from 2v1 training

- With a lost aircraft costing only −0.7, the team learned to sacrifice the lead as bait and kill with the wingman: 90% mutual kills. The cost is now −1.0, and red also shoots the second aircraft.
- UCAVs are slower than F-16s. With a 300 s limit, even ideal play timed out on most starts beyond 85 km. Raise MAX_STEPS (400–480) for slow platforms.

15 · The parameter library

Every number the simulation uses for an aircraft, its radar and its missile is stored in the **library**, as JSON files.

Kind	Holds	Built-in items
airframe	Mass, wing, thrust, drag, limits	F-16C, GENERIC-UCAV
radar	Range, field of view, error sizes	APG-68, GENERIC-UCAV-RADAR
missile	Motor, drag, guidance, seeker, hand-off, support rule	AIM-120
platform	An airframe + radar + missile, missile count, RCS, speed choices, altitude band	F-16C, GENERIC-UCAV

Folders

- `library/builtin/<kind>/:` shipped with the code, read-only.
- `library/user/<kind>/:` your copies.
- `library/envelopes/:` calibrated missile envelopes, one per missile version.

Making your own aircraft, step by step

- 1 On the LIBRARY tab, open a built-in item and click  **Copy...** Built-ins are never changed.
- 2 Edit the copy's values and click  **Save**. Each value has a unit, a valid range and a source note.
- 3 For an airframe or platform, click  **Performance card**: it flies the flight model and reports top speed, turn, climb, ceiling and autopilot behaviour, with warnings.
- 4 For a missile, click  **Calibrate envelope** (or `python sweep_envelope.py --missile <id>`). Training refuses to start until a missile has an envelope for its exact values. The file name carries a fingerprint, so editing the missile invalidates the old table. A table calibrated before SIM_REV 11 has no target altitude and assumes a level target; training says so at the start. Recalibrate it.
- 5 Select your platform under ENGAGEMENT on the TRAINING page.

Old models keep their meaning

Every checkpoint stores the platforms and the full items it was trained with. Later edits in the library never change what an old checkpoint means.

PART C

The GUI, page by page

Every field and every number on the four pages, and what good values look like.

- .6. The TRAINING page
- .7. The WATCH page
- .8. The CROSS-PLAY page
- .9. The LIBRARY page

16 · The TRAINING page

16.1 Setup fields

Field	Default	What it does
Engagement	1 v 1, F-16C v F-16C	Format, the agent's platform, the opponent's platform (and the wingman's in 2v1).
Opponent stage	STRAIGHT	The curriculum stage to start at.
Missile doctrine	MIXED	The cost per missile: one fixed doctrine, or a random one per episode.
Training steps	2,000,000	Decisions to train for. The estimate shows the expected time.
Parallel envs	1	Simulations run at once, one CPU core each.
Gamma · Learn rate · Batch	0.997 · 0.00025 · 256	PPO settings (chapter 11).
Auto-curriculum	ON	Advance through the stages automatically. OFF keeps the starting stage.
Model name	latest	File the final model is saved as.
Resume from	fresh start	Continue from a saved model.
Advance at · Min episodes · Hold for	65% · 60 · 5	The curriculum rule (chapter 12).
Self-play (advanced)	33% · 50% · 250k · 12	Chapter 13. Greyed out when the run can't reach self-play.

The warning line under ENGAGEMENT explains consequences of the matchup, for example that different platforms end the curriculum at ADAPTIVE.

16.2 Key numbers (over the last 50 episodes)

Number	Meaning	Good looks like
Win rate	Share of episodes won outright.	Rising. 65% advances the curriculum.
Exchange ratio	(kills + mutual kills) ÷ (losses + mutual kills): enemies destroyed per own aircraft lost. With no losses, the number of enemies destroyed.	Above 2 is good; about 1 is trading evenly.
Realised Pk	Kills per missile fired.	0.25–0.45 against armed opponents is typical here.
Timeout rate	Share of episodes with no result.	Low. High means hesitating, or episodes too short.
Launch range	Average range at launch.	Depends on the platforms; compare with $r/R_{\max}$.
$r / R_{\max}$ at shot	Where in the envelope it shoots.	0.6–0.85. Above 0.9 is shooting too far out.
Shots / episode	Missiles fired per episode.	1–3. Very high means spraying.
Track fraction	Share of seconds with a firm track.	Rising; 70–80% in a good run.

Bank reversals / min	Wing-rocking check.	A few a minute. 15+ means switching heading choice too often.
Blue lost / episode	2v1 only.	Low.

16.3 Charts

- **Outcomes:** win, loss, timeout and mutual shares over training. The dashed line is the advance mark.
- **Exchange ratio, realised Pk, shots / episode, $r / R_{\max}$ at shot, track fraction:** trends of the key numbers.
- **Bank reversals / min,** with a line underneath: heading-choice changes per minute and the most frequent pairs, e.g. "+30 ↔ +50° 42%". It tells you whether the agent rocks left–right or only adjusts on one side.
- **Reward drivers:** how much each shaping term and cost moved the reward recently.
- **By doctrine:** shots, Pk and win rate under each doctrine.
- Each chart has a maximise button. The history is thinned evenly, so long runs keep their whole shape.

16.4 Curriculum bar and CURRENT RUN

- The bar shows the stage, the win rate against the advance mark, and progress, e.g. "advance at 65% → EVASIVE (held 3/5 checks)".
- **CURRENT RUN** lists the running configuration and startup notes, such as warm-start notes when resuming an older model (chapter 22) and "self-play pool cleared".

17 · The WATCH page

17.1 Fields

Field	What it does
Opponent	STRAIGHT ... ADAPTIVE, or SELF PLAY (snapshots from the pool).
Playback speed	0.5× to 50×.
Episodes	Stop after 10, 25, 50, 100, 200, or play until stopped.
Missile doctrine	The cost per missile the model is told about.
Opponent flies · Format	Default: as the model was trained.
Checkpoint to load	The model to watch, or a random policy.

17.2 Watch statistics

Summed over the episodes since START EVAL; episodes stopped part-way are left out.

- **Outcome counts** and shares.
- **Win rate with its 95% range,** e.g. "40% (22–61%)". With 10 episodes the range is wide, so don't read much into it.
- **Exchange ratio, realised Pk, shots / episode, mean $r/R_{\max}$ at shot, mean length, timeout rate;** in 2v1, blue lost per episode.
- **By start geometry and by start range:** win / mutual / loss / timeout / other for each. ⇔ marks mirrored starts, where the opponent had the better start. This shows *where* the model wins and loses.

17.3 3-D view and shot trace

- Left-drag rotates, right-drag pans, the scroll wheel zooms. Aircraft stay vivid at any zoom.
-  **Shot Trace** (log bar) lists each launch of the last episode with range, aspect, track state, uncertainty, track age and $r/R_{\max}$.

18 · The CROSS-PLAY page

18.1 Fields

Field	What it does
Checkpoints	Models to include, with quick links for all, archive only, or none.
Scripted columns	Add scripted opponents as extra columns.
Scripted platform	What scripted opponents fly (default: each model's training opponent).
Games per cell	Games per ordered pairing; 100 is a good default.
Workers	Parallel processes.
Doctrine	Doctrine for both sides.

18.2 How games are scored

- Win = KILL or BANDIT_CRASH (1). Loss = SHOT_DOWN or CRASH (0). Draw = MUTUAL_KILL, TIMEOUT or ESCAPE (0.5).
- The row model flies AC1; the column model AC2, with its own radar and track.
- Game i of every pairing starts from the same geometry. Both orders (A v B and B v A) are played on the same starts.
- Models take their most likely action (sampling can be switched on from the command line).

18.3 Seat bias

A model against itself should score 0.5. The difference is the AC1 seat's advantage. Starts used to be drawn from AC1's side, which gave AC1 a real head start (a bias of about +0.13). Since half the starts are now mirrored, the bias should sit near 0. Either way it cancels in head-to-head scores and Elo, because every pairing is played from both seats. A large bias usually means few games per cell.

18.4 Elo

A rating for each model and scripted opponent, fitted over all games, mean 1000. 100 points $\approx$ a 0.64 expected score for the higher-rated side.

19 · The LIBRARY page

- **Left:** the items by kind, built-in and yours.
- **Right:** the selected item's values grouped by topic, each with unit, valid range and source. Advanced groups are folded.
-  **Copy...** makes an editable copy of any item.  **Save** checks every value against its valid range;  **Delete** removes one of your items.
-  **Performance card** (platforms and airframes): flies the flight model and reports what it can do.
-  **Calibrate envelope** (missiles): measures the launch envelope. A missile can't be used in training without it. Calibrated missiles show  cal in the list.

PART D

Working with results

How to judge a run, what to do when something looks wrong, where everything is stored, and reference material.

- !0. Reading a training run
- !1. Symptoms and what to do
- !2. Files, checkpoints and versions
- !3. Command-line reference
- !4. DCS World: first step
- !5. Troubleshooting
- !6. Glossary

20 · Reading a training run

A healthy run

- **Track fraction rises first.** The agent learns to point its radar at the enemy before it learns to shoot well.
- **Win rate rises and timeouts fall** at each stage, then **dip after each advance** and recover. A dip is normal: the new opponent is harder.
- **$r/R_{\max}$ at shot settles in 0.6–0.85**, and Pk rises.
- **Bank reversals stay at a few a minute.**

Judging progress fairly

- **Training numbers move with the opponent.** A lower win rate after an advance, or after a change such as mirrored starts, doesn't mean a worse model.
- **Compare models in CROSS-PLAY**, on the same starts, from both seats, many games.
- **Use WATCH statistics** to see *where* a model wins and loses: by start geometry and range.
- **Watch some episodes.** Several problems in this project were found by watching: the dive in 2v1, wing-rocking, hesitation at the start.

TensorBoard

Training also writes to `tb_logs_bvr/`. Start TensorBoard with `tensorboard --logdir tb_logs_bvr` for more detail: PPO internals (clip fraction, KL), every reward term, the win rate split by opponent kind in self-play, and heading switches per minute.

PPO health check

If the clip fraction stays around 0.25 or higher, and the KL around 0.02 (the stop threshold), updates are large. Lowering the learning rate (e.g. to 0.0001) often makes training steadier.

21 · Symptoms and what to do

Each row comes from something observed while developing this project.

Symptom	Likely cause	What to do
Many timeouts, especially from long starts	Slow platforms can't close the distance in 300 s. With UCAVs (250 m/s) even ideal play timed out on most starts beyond 85 km.	Raise <code>MAX_STEPS</code> in <code>bvr_env.py</code> (e.g. 400–480). Check WATCH statistics by start range.
Curriculum stuck at a stage	An outmatched platform may never reach 65%.	Lower <code>ADVANCE AT</code> for that matchup; check whether the stage is winnable by watching.
Curriculum advances, then collapses	Advanced on a lucky streak.	Keep <code>HOLD FOR</code> at 5 or more; use <code>MIN EPISODES 60+</code> .
Bank reversals 15+ a minute	The policy keeps switching heading choice.	Read the heading-pair line under the chart. Left ↔ right pairs are rocking; same-side pairs are crank adjustments. The heading cost is 0.04; a fresh run learns it better than a resumed one.

Shots at $r/R_{\max}$ above 0.9, low Pk	Shooting at the edge of the envelope; the target turns away and the missile runs out of energy.	Expect it to improve with training; try the BALANCED or CONSERVATIVE doctrine.
Low track fraction against a small target	Radar range shrinks with target size: about 38 km for a UCAV radar against a UCAV.	Expected; longer episodes help the agent close.
2v1: mostly mutual kills	Sacrificing one aircraft paid off.	Already addressed (−1.0 per loss, red targets the nearer aircraft and shoots both).
Win rate drops after resuming	The model was trained under an older simulation revision.	Read the warm-start notes in CURRENT RUN (chapter 22); give it time.
Win against snapshots rises, against scripted falls	Self-play overfitting.	Raise the self-play scripted share.
Large seat bias in cross-play	Few games per cell.	Add games; it never skews head-to-head scores or Elo.

22 · Files, checkpoints and versions

Where things are

Path	Contents
<code>models_bvr/<name>.zip</code>	A run's final model, saved when the run ends or is stopped.
<code>models_bvr/archive/pre_<STAGE>.zip</code>	The model just before the curriculum advanced to STAGE.
<code>models_bvr/archive/best_<STAGE>_<rate>.zip</code>	The best win rate reached at a stage: your backup if training later makes the model worse.
<code>models_bvr/selfplay_pool/</code>	Self-play snapshots of the current run. Emptied at the start of every run.
<code>tb_logs_bvr/</code>	TensorBoard logs, one folder per run.
<code>bvr_metrics.json</code>	What the dashboard shows, rewritten every 10 episodes.
<code>crossplay_results/</code>	Cross-play matrices, Elo and raw games.
<code>library/</code>	Parameter library (chapter 15).

Simulation revisions and warm starts

When the simulation, inputs or rewards change in a way that changes what a trained model experiences, the simulation revision goes up. Resuming a model trained under an older revision works, but the trainer prints a **warm-start note** for each change it missed, so you know it isn't a pure continuation. Older models also get new inputs added automatically, with zero weight, so they keep working.

Rev	Change
2	Missile time-to-go fixed; time-to-go inputs, defence reward and scripted defence changed.
3	Defence term became $-0.8 \times (1 - \tanh(\text{time-to-go} / 25))$: an inbound missile only ever lowers the score.
4	Without a track, the heading choice is an offset from the last known bearing; 4 heading inputs added.
5	2v1: red opens on the nearer aircraft and shoots both; a lost blue aircraft costs −1.0.
6	Missiles only within 60° of the nose.
7	Heading change costs 0.04 per 180° (was 0.015).

23 · Command-line reference

Everything the GUI does can also be run from a terminal.

`train_bvr.py`: training

```
python train_bvr.py --steps 2000000 --n-envs 8 --opponent straight --run-name f16_first
```

Option	Default	Meaning
<code>--steps</code>	2,000,000	Decisions to train.
<code>--n-envs</code>	1	Parallel environments.
<code>--opponent</code>	straight	Starting stage: straight, evasive, notcher, shooter, adaptive_shooter, self_play.
<code>--resume PATH</code>	none	Continue from a saved model.
<code>--run-name</code>	latest	Final model file name.
<code>--save-dir</code>	models_bvr	Where models go.
<code>--no-curriculum</code>		Stay on the starting stage.
<code>--advance-win-rate</code>	0.65	Curriculum mark.
<code>--advance-min-episodes</code>	60	Episodes at a stage before advancing.
<code>--advance-hold-checks</code>	5	Checks in a row at the mark.
<code>--sp-scripted-frac · --sp-newest-frac</code>	1/3 · 0.5	Self-play opponent mix.
<code>--sp-snapshot-steps · --sp-pool-size</code>	250,000 · 12	Self-play snapshots.
<code>--doctrine</code>	mixed	mixed, aggressive, balanced, conservative.
<code>--platform · --opponent-platform</code>	F-16C	Library platforms.
<code>--format · --wingman-platform</code>	1v1	1v1 or 2v1; the wingman's platform.
<code>--gamma · --lr · --batch-size</code>	0.997 · 0.00025 · 256	PPO settings.
<code>--seed</code>	42	Random seed.
<code>--no-privileged</code>		Critic without the true values.

`crossplay.py`: comparing models

```
python crossplay.py models_bvr/archive --episodes 100 --workers 8 --scripted shooter adaptive_shooter
```

Options: `--episodes` (games per pairing), `--scripted`, `--scripted-platform`, `--doctrine`, `--stochastic` (sample actions), `--workers`, `--seed`, `--out`.

Other tools

Command	What it does
<code>python bvr_gui.py</code>	Starts the GUI (<code>--port</code> , <code>--ws-port</code> , <code>--model-dir</code>).
<code>python test_sim.py</code>	Runs the 36 tests.
<code>python sweep_envelope.py --missile ID</code>	Calibrates a missile's launch envelope (<code>--quick</code> for a coarse check).
<code>python bvr_perf.py PLATFORM</code>	Performance card of a platform.
<code>python dcs_obs_check.py LOG.jsonl</code>	Compares a DCS recording's inputs with training (chapter 24).

24 · DCS World: first step

The long-term aim is to fly a trained model in DCS World. Training stays in this simulator, since DCS runs in real time and has no quick reset. The first step, already built, checks whether a policy's inputs would look the same in DCS as in training.

- 1 **Allow file access** for mission scripts: in `<DCS>\Scripts\MissionScripting.lua`, comment out `sanitizeModule('io')` and `sanitizeModule('lfs')`. While they are off, any mission can read and write files on your computer; restore them afterwards.
- 2 **Build a mission** with one blue and one red jet, 60–110 km apart, and a MISSION START trigger running `dcs/bvr_logger.lua` (DO SCRIPT FILE).
- 3 **Fly it**. The logger writes `Saved Games\DCS\Logs\bvr_rl_*.jsonl`, 10 times a second.
- 4 **Check it**: `python dcs_obs_check.py <file> --platform F-16C --opp-platform F-16C --max-steps 400`. Inputs flagged OUTSIDE or CLIP are ones the policy has rarely or never seen.

Details in `dcs/README.md`. Only 1v1 so far. The logger has been tested against a mock of the DCS scripting API, not yet inside DCS.

25 · Troubleshooting

Problem	What to check
GUI page doesn't open	Is <code>bvr_gui.py</code> running? Open http://localhost:5006 . If a port is busy, start with <code>--port / --ws-port</code> .
Dashboard stays empty	It fills after 10 episodes. The first may take 30–60 s.
A model isn't in the list	The list is read from <code>models_bvr/</code> when a run or eval stops; reload the page.
Changes after <code>git pull</code> don't appear	Restart the GUI. A running training keeps its old code until the next run.
"SELF_PLAY needs both sides on one platform"	Self-play only works with the same platform on both sides, in 1v1.
Training refuses: missile not calibrated	Calibrate the missile (chapter 15).
Warm-start notes when resuming	Normal: the model was trained under an older revision (chapter 22).
Warning on the TRAINING page about code or files	The GUI reports local edits or library problems. Read the message; it names the file.
Training is slow	Add parallel envs (one per spare core). Stop WATCH playback while training: it uses CPU too.

A test fails

Run `python test_sim.py` again to see the failing check and its message; pull the latest code.

26 · Glossary

Term	Meaning
BVR	Beyond visual range: combat by radar and radar-guided missiles at tens of kilometres.
Agent · policy · model	The neural network that flies the aircraft. A saved one is a checkpoint (.zip).
Episode	One engagement, from start to outcome.
Step · decision	One choice by the agent: one simulated second.
Bandit	The enemy aircraft.
Aspect	Which way the target points relative to you: 0° head-on, 180° tail-on.
Off-boresight	How far the target is off your nose.
Crank	Turning 30–50° off the bandit after a launch: keeps it on radar while slowing the closure.
Beam · notch	Flying across the enemy's line of sight (90°). It lowers the closure the enemy's radar sees and can degrade its track.
Drag	Turning away and running.
$R_{\max}$ · R_{NEZ}	The longest range a missile can hit a target that keeps flying as it is · the range inside which it hits even if the target runs.
Pk	Probability of kill: kills per missile fired.
Support · hand-off	Guiding your missile with radar updates · the moment its own seeker takes over.
Pitbull	The missile's seeker is active; it no longer needs support.
RWR	Radar warning receiver: tells you when a missile is launched at you.
RCS	Radar cross-section: how big a target looks on radar (m ²).
Track · coast	The radar's estimate of the target · continuing that estimate without new reports.
Datalink	Sharing tracks between aircraft (2v1) or guiding a missile.
Doctrine	The cost per missile the agent is told about: aggressive, balanced, conservative.
PPO	Proximal Policy Optimization: the learning method.
Actor · critic	The half of the network that chooses · the half that judges situations.
Shaping · potential	Small rewards every second · the score of a situation they are computed from.
Mask	Removing impossible choices before the network chooses.
Curriculum	Opponents of increasing difficulty.
Self-play · snapshot · pool	Training against copies of itself · a frozen copy · the set of copies.
Cross-play · Elo	Playing saved models against each other · a rating from those games.
Seat · seat bias	AC1 or AC2 in a game · how much better the AC1 seat scores on the same starts.
Mirrored start	A start in which the two aircraft swap positions, so the opponent gets the better start.
Simulation revision · warm start	The version number of what a model experiences · continuing an older model under a newer revision.